Swiss Institute of Bioinformatics

Introduction to Machine Learning techniques

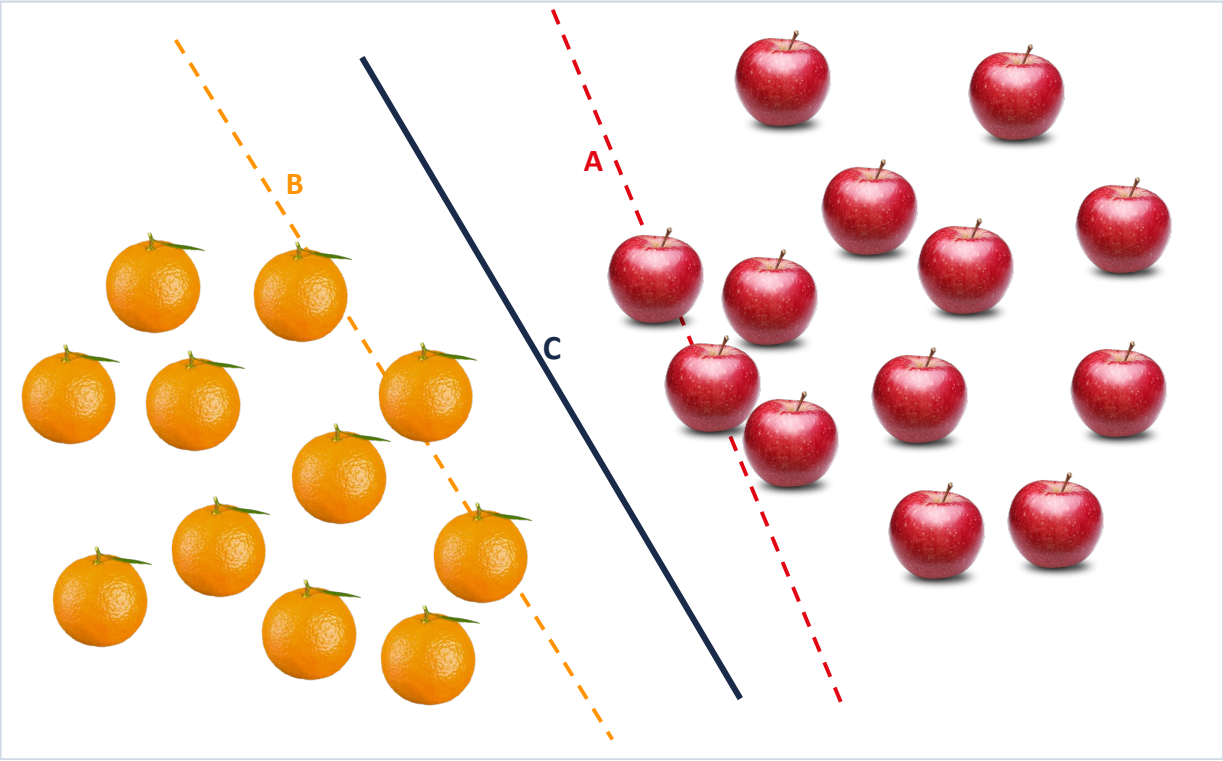
SIB Training Course

March 2026

SUPPORT VECTOR MACHINE – CORE IDEA

SVM: Drawing the safest line

Three candidate separating lines: which is the best?



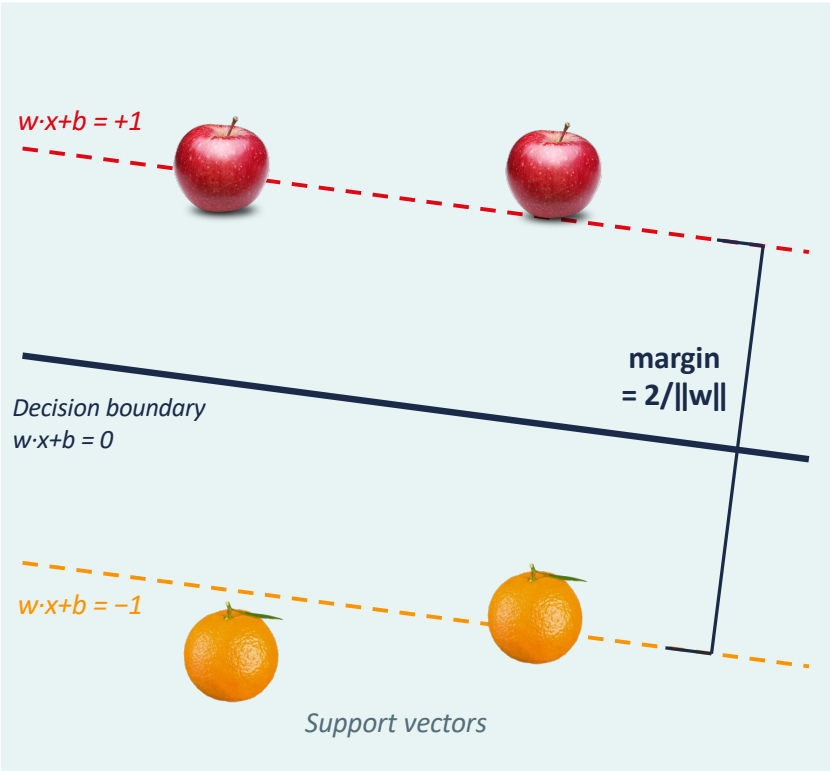
- Line B → dangerously close to the cluster
- Line C → maximum breathing room on both sides

breathing room := margin

- Larger margin ~ more robust to new data
- SVMs find line C automatically

SVM CONCEPT

The margin: Measuring the breathing room



What is the margin?

- The margin := the gap between the two classes around the decision boundary
- $2/\|w\|$ inversely proportional to the weight vector length
- Wider margin → more confident classification

What are support vectors?

- Training points touching the dashed lines → 'support' the boundary position
- Remove a non-support vector → boundary unchanged
- Remove a support vector → boundary shifts

HARD-MARGIN SVM

The optimization: Finding the widest margin

Goal: maximize margin $2/\|\boldsymbol{w}\|$ ≡ minimize $\frac{1}{2}\|\boldsymbol{w}\|^2$

Primal problem

minimize $\frac{1}{2}\|\boldsymbol{w}\|^2$

over $w \in \mathbb{R}^d,\ b \in \mathbb{R}$

subject to:

$$\boldsymbol{y}_i\,(\boldsymbol{w}^T\boldsymbol{x}_i + \boldsymbol{b}) \geq \mathbf{1} \quad \textbf{for all } \boldsymbol{i}$$

← *one constraint per training point*

Decoding the formula:

$\frac{1}{2}\|\boldsymbol{w}\|^2$

Shrinking ‖w‖ widens the margin.

$\boldsymbol{y}_i \in \{-\mathbf{1}, +\mathbf{1}\}$

Labels are +1 and −1.

$\boldsymbol{y}_i(\boldsymbol{w}^T\boldsymbol{x}_i + \boldsymbol{b})$

Signed distance of point i from the boundary. Positive = correct side.

$\geq \mathbf{1}$

Every point must be outside the margin band, not just correctly classified.

→ *Convex QP: guaranteed unique global solution. No local minima to get trapped in.*

LAGRANGIAN DUALITY

The dual: A smarter way to solve it

Reformulates the problem so the answer only depends on dot products between training points → unlock kernels

Dual problem

maximize $\Sigma_i \alpha_i - \frac{1}{2} \Sigma_i \Sigma_j \alpha_i \alpha_j y_i y_j x_i^T x_j$

subject to: $\alpha_i \geq 0$ and $\Sigma_i \alpha_i y_i = 0$

Solution: $w^* = \Sigma_i \alpha_i y_i x_i$ **(only $\alpha_i > 0$ count)**

Predict: $f(x) = \text{sign}(\Sigma_i \alpha_i y_i x_i^T x + b)$

Only dot products matter

The problem only involves $x_i^T x_j$, NOT the points individually.

Sparsity for free

Most $\alpha_i = 0$. Only the support vectors ($\alpha_i > 0$) define the boundary. Efficient prediction.

Duality is exact

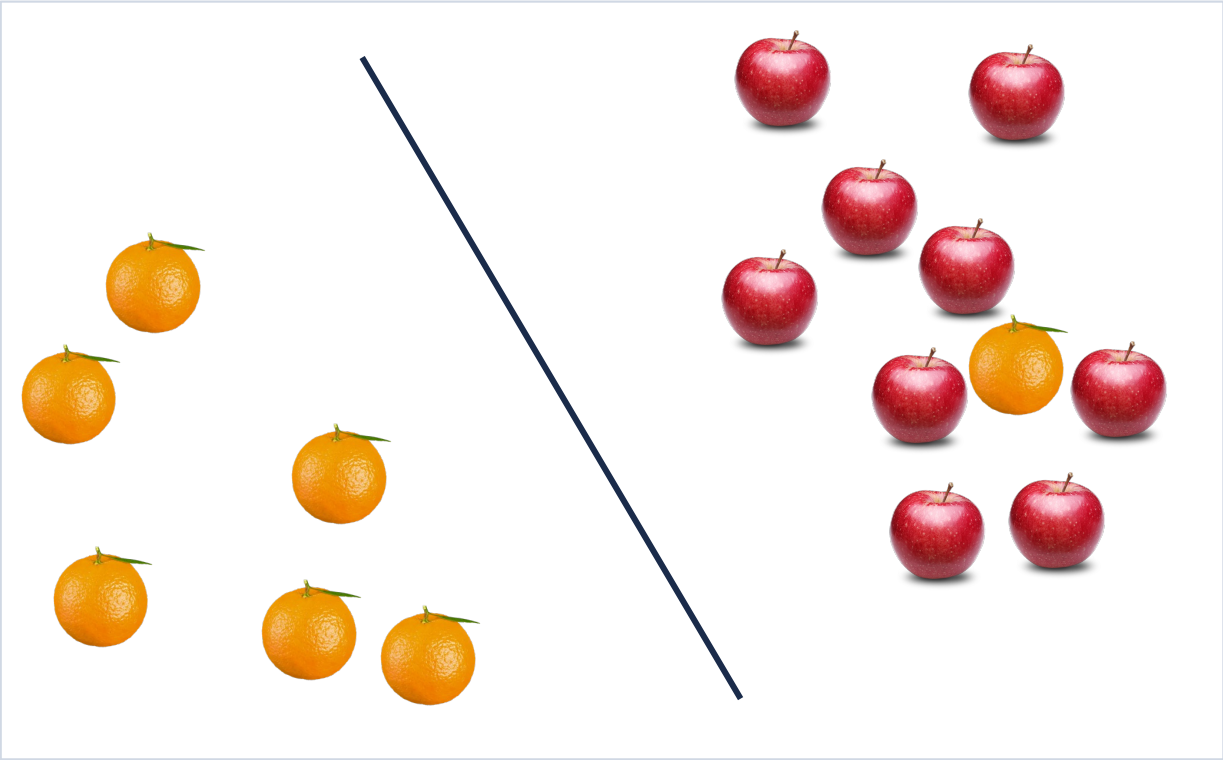
Primal and dual have the same optimal value.

Karush-Kuhn-Tucker condition: $\alpha_i [y_i(w^T x_i + b) - 1] = 0$
→ a point is either on the margin boundary (support vector) or ignored ($\alpha_i = 0$).

REAL-WORLD SVM

Soft margin: When data are not perfect

Soft margin: let some points cross the boundary but penalize them.



New objective:

$$\min \ ½\|\boldsymbol{w}\|^2 \ + \ \boldsymbol{C} \cdot \Sigma_i \, \xi_i$$

margin maximization + total violation penalty

- $\boldsymbol{C} = \infty$

→ hard margin (no violations tolerated)

- **Large $\boldsymbol{C}$**

→ narrow margin, few errors (overfitting risk)

- **Small $\boldsymbol{C}$**

→ wide margin, some errors (underfitting risk)

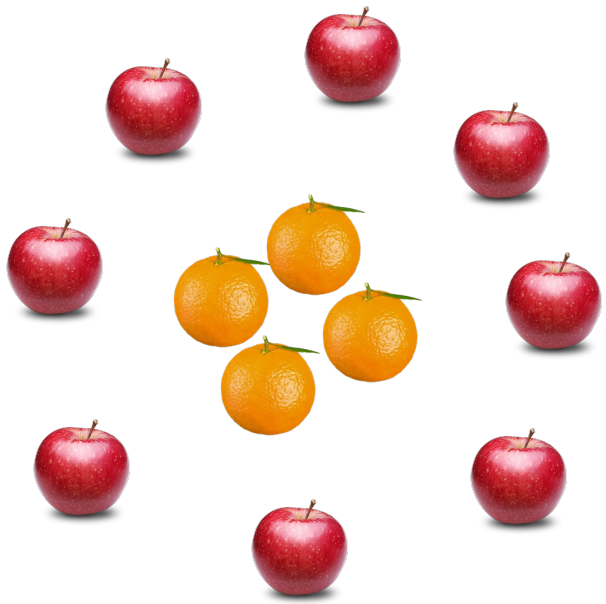
- $\boldsymbol{C}$ is the 1st hyperparameter to tune

with cross-validation

KERNELS — CORE CONCEPT

The kernel trick: Lifting data to separability

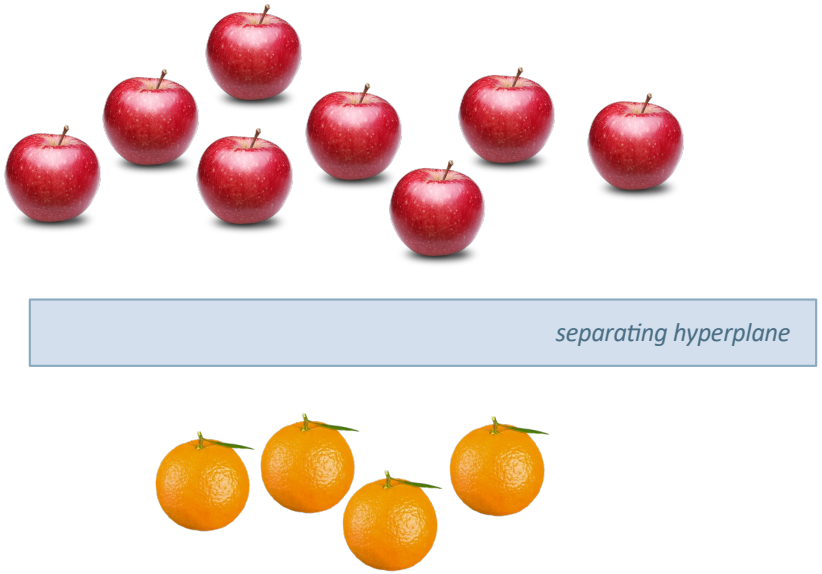
2D input space: not separable



✗ **No straight lines separate these**

$$\xrightarrow[\text{lift!}]{\phi(x)}$$

Feature space: now separable!



✓ **Flat plane separates perfectly**

*$K(x_i, x_j)$ computes the **dot product** in the lifted space without ever computing $\phi(x)$ explicitly: computational savings.*

KERNELS — INTUITION

Common kernels: What they measure

Kernel $K(x, z)$: how similar are x and z in feature space?

Linear: $K(x, z) = x^{\mathrm{T}} z$

"Are x and z pointing in the same direction?"

Use when: features already have meaning (e.g. TF-IDF text vectors). No new feature space — works directly in input space.

Example: spam detection - word count vectors. Two emails with similar word patterns get high K.

Polynomial: $K(x, z) = (x^{\mathrm{T}} z + c)^{\mathrm{d}}$

"Do x and z share combinations of features?"

Use when: feature interactions matter (e.g. image patches). Degree d controls how complex the interactions are.

Example: d=2 adds all pairwise products $x_i x_j$ automatically — like adding $x_1 x_2$, $x_1 x_3$, … as new features.

RBF / Gaussian: $K(x, z) = \exp(-\gamma \|x - z\|^2)$

"How close are x and z in space?"

Use when: decision boundary is complex, unknown shape. Most popular default, works in infinite-dimensional space.

Example: K → 1 when x ≈ z (nearby points), K → 0 when far apart. γ controls how quickly similarity drops off.

Sigmoid: $K(x, z) = \tanh(\kappa x^{\mathrm{T}} z + \theta)$

"Do x and z activate a neuron similarly?"

Use when: mimicking a neural network layer.

Example: behaves like a single-hidden-layer neural net.

PRACTICAL GUIDE

Practical SVM: When and How to use

When SVMs shine:

✅ High-dimensional data

Text, gene expression: many features, moderate samples

✅ Small to medium datasets

< 100K points; deep learning shines with more data

✅ Clear margin of separation

When classes are well-separated in feature space

✅ Need theoretical guarantees

Convexity, no local minima

When to prefer other models:

⚠️ Very large datasets

Slow: Kernel SVMs scale as $O(n^2)$ to $O(n^3)$

⚠️ Multi-class tasks

Needs OvR/OvO wrappers; less natural than softmax

⚠️ Needs feature scaling

Must normalize features before training

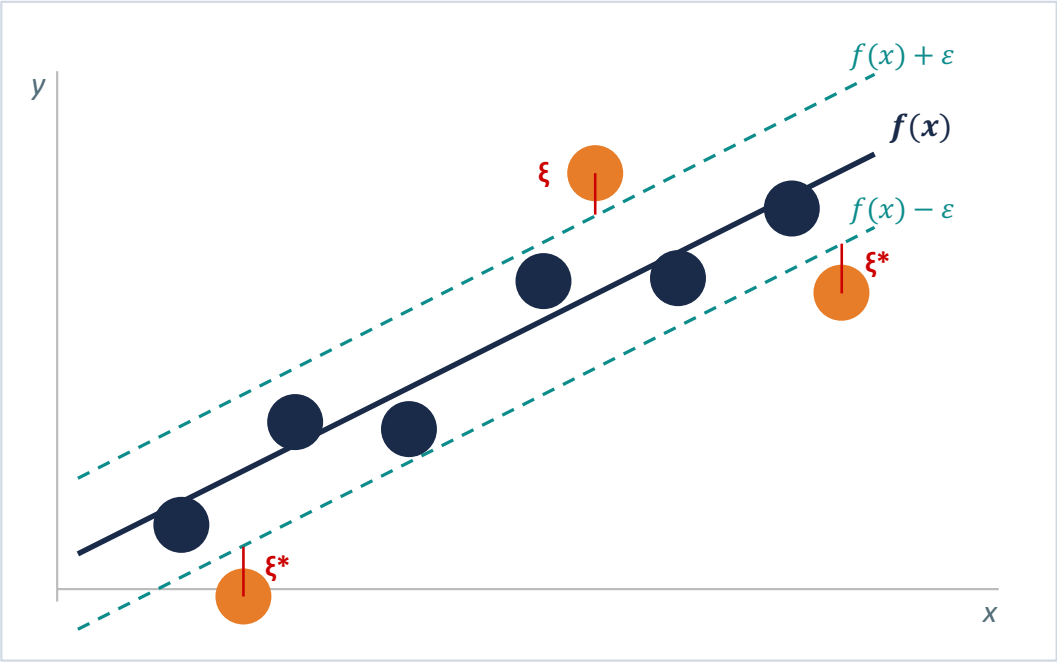
⚠️ Black-box kernels

Probability outputs require Platt scaling

SUPPORT VECTOR REGRESSION (SVR)

SVM for Regression: Flipping the idea

Classification: keep points outside the margin tube
Regression: keep points inside a tube around the prediction



● *inside tube = no penalty* ● *outside = penalty ξ**

ε-insensitive loss

$$L(y, f(x)) = \max(0, |y - f(x)| - \varepsilon)$$

Zero loss inside the tube. Linear penalty outside.

SVR objective

$$\min \tfrac{1}{2}\|w\|^2 + C\,\Sigma_i(\xi_i + \xi_i^*)$$

$\xi_i \geq 0$ *: above tube,* $\xi_i^* \geq 0$ *: below tube*

ε	**Tube width:**	Controls how much error is free. Larger ε → simpler model, more bias.
C	**Penalty weight:**	Large C punishes violations
K	**Kernels**	Lift to higher-dimensional space → nonlinear regression

SVR — THEORY

SVR Dual & Support Vectors: Towards kernels

Support vectors define the fitted curve. Everything else is forgotten.

SVR Dual problem

$$\max \; -\tfrac{1}{2} \Sigma_{ij} (\alpha_i - \alpha_i^*)(\alpha_j - \alpha_i^*) K(x_i, x_j) - \varepsilon \Sigma_i(\alpha_i + \alpha_i^*) + \Sigma_i y_i(\alpha_i - \alpha_i^*)$$

subject to:

$$\Sigma_i(\alpha_i - \alpha_i^*) = 0$$
$$0 \le \alpha_i, \alpha_i^* \le C$$

Prediction: $f(x) = \Sigma_i (\alpha_i - \alpha_i^*) K(x_i, x) + b$

Sparsity: most $\alpha_i = \alpha_i^* = 0$

Points inside the tube contribute nothing. Only tube-edge and outside points contribute.
→ Efficient prediction

Two alphas per point (α_i and α_i^*)

$\alpha_i > 0$ above the tube. $\alpha_i^* > 0$ below. At most one of the pair is non-zero for any given point.

C caps each α: $0 \le \alpha_i, \alpha_i^* \le C$

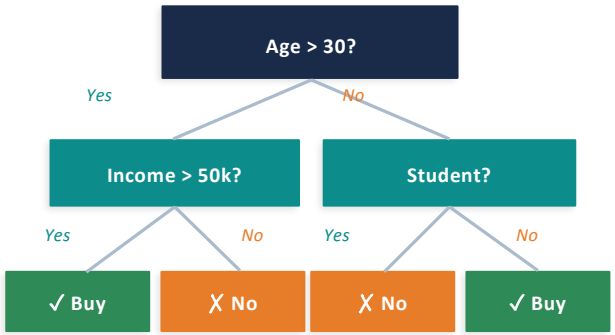
Large C → penalises tube violations heavily → fewer support vectors escape.

Karush-Kuhn-Tucker condition: $(\alpha_i - \alpha_i^*)$ is non-zero only when $|y_i - f(x_i)| \ge \varepsilon$
→ confirming that only tube-edge & outside points (support vectors) are considered

DECISION TREES — CORE IDEA

Decision Trees: Learning by asking questions

Game '20 Questions'



Example: Will a customer buy?

Recursive partitioning

At each node, find the feature & threshold that best splits the data. Repeat inside each partition until a stopping rule fires.

Fully interpretable

Able to trace any prediction top-to-bottom. No black box → easy to explain

Handles mixed data natively

Works with continuous and categorical features, missing values, and different scales: no normalisation needed.

Prone to overfitting

A **deep** tree memorises training data perfectly but generalises poorly.

DECISION TREES — SPLITTING

How Trees split: Gini impurity & Information gain

At every node, which split makes the resulting groups most pure?

Gini Impurity

$$Gini = 1 - \Sigma_k p_k^2$$

p_k = fraction of class k in the node

Before split — impure (Gini ≈ 0.5):

After best split — purer (Gini ≈ 0.0):

Gain = Gini(parent) − Σ weighted Gini(children)

Information Gain (Entropy)

$$H(S) = -\Sigma_k p_k \log_2(p_k)$$

Gain = H(parent) − Σ weighted H(children)

Pure node (all one class):	H = 0 → no surprise
Perfectly mixed (50/50):	H = 1 bit → maximum uncertainty
Best split maximises:	Gain = how much H drops after splitting

The split threshold is found by sorting each feature and testing every midpoint.

DECISION TREES — REGULARISATION

Overfitting & Pruning: Keeping trees honest

Too shallow

High bias, low variance

Underfit - missing real patterns in the data. Training and test error both high.

max_depth = 1–2

Just right

Balanced bias-variance

Generalise well - capturing real structure without memorising noise.

max_depth = 5–10 (tune with CV)

Too deep

Low bias, high variance

Overfit - memorising training noise. Low training error, high test error.

max_depth = unlimited

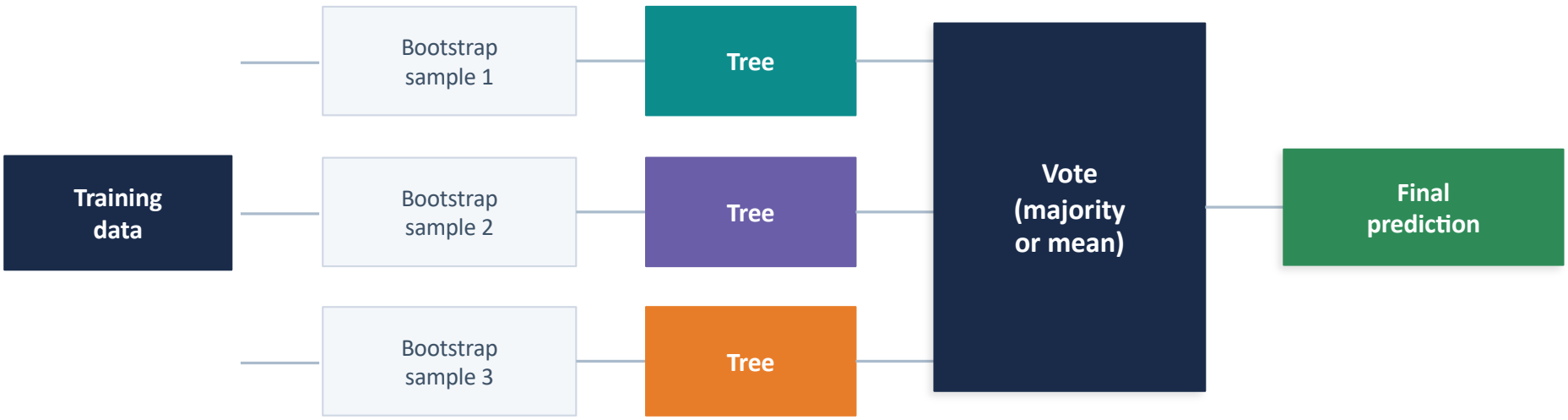
How to regularise a decision tree:

max_depth	Hard ceiling on tree height.
min_samples_leaf	Every leaf must have at least N samples. Smooths out noisy leaves.
min_samples_split	Do not split a node with fewer than N samples. Prevents tiny, over-specific splits.
cost-complexity pruning (ccp)	Grow full tree, then remove branches that add little accuracy vs. complexity.

RANDOM FORESTS — CORE IDEA

Random Forests: Wisdom of the crowd

Random forests := many deliberately different trees => majority vote.



Sources of randomness:

Bagging (Bootstrap Aggregating)

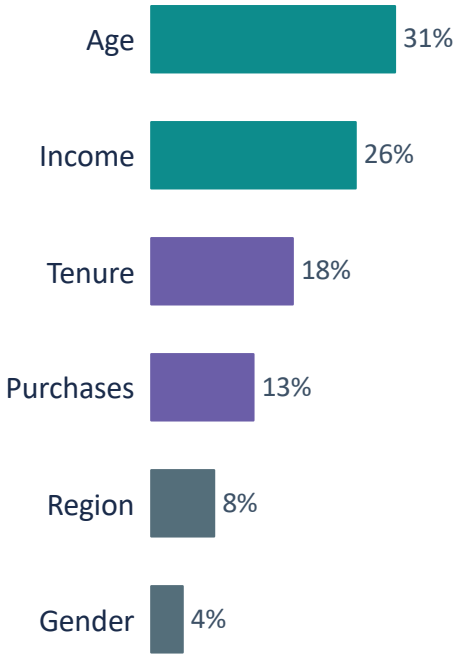
Each tree trains on a bootstrap sample (~63% of data).
Different trees see different data → make different mistakes.

Random feature subsets

At each split, only subset of features are considered (not all). Prevents all trees from using the same dominant feature → true diversity.

Random Forests: Feature importance & Practical tuning

Feature importance (mean Gini decrease)



How much each feature reduces impurity on average across all trees.

Out-of-bag (OOB) error

Each tree trains on ~63% of data
The remaining ~37% = OOB samples -> built-in validation

Key hyperparameters:

n_estimators	More trees = more stable. No overfitting risk from adding trees.
max_features	Lower = more diversity but weaker individual trees.
max_depth	Limiting can speed training with minor accuracy loss.
min_samples_leaf	Ensuring large leaves

RF for Regression: What matters

CLASSIFICATION

Discrete output
Vote
Split: Gini impurity, Entropy reduction

Example: Is this fruit an orange or an apple?

REGRESSION

Continuous output
Average
Split: variance reduction, MSE reduction

Example: How much does this fruit cost?